

Documentație Proiect Machine Learning

Signal Object Detection

Popescu Iulia-Maria

9 iunie 2026

Cuprins

1	Introducere	3
2	Preprocesarea Datelor și Reprezentarea Trăsăturilor	3
2.1	Pentru Modelele Clasice (Naive Bayes și SVM)	3
2.2	Pentru abordarea cu Random Forest (HOG)	3
3	Modele de Machine Learning Clasice	4
3.1	Gaussian Naive Bayes (Rezultat Kaggle: 0.27127)	4
3.2	Support Vector Machine (SVM) (Rezultat Kaggle: 0.25890)	4
3.3	Random Forest pe caracteristici HOG (Rezultat Kaggle: 0.35418)	4
4	Rețele Neuronale Convoluționale (CNN)	5
4.1	Varianta 1: CNN de Bază (Rezultat Kaggle: 0.50545)	5
4.2	Varianta 2: Rafinarea Clasicatorului (Rezultat Kaggle: 0.51418)	5
4.3	Varianta 3: CNN Profund (Rezultat Kaggle: 0.63636)	5
4.4	Varianta 4: Tehnici State-of-the-Art (Rezultat Kaggle: 0.72363)	6
4.5	Varianta 5: Fine-Tuning și Class Weights (Rezultat Kaggle: 0.74545)	6
5	Investigarea de Arhitecturi și Ajustări Adiționale (Variantele 6, 7 și 8)	7
5.1	Varianta 6: Penalizarea Clasei Majoritare (Scor: 0.48363)	7
5.2	Varianta 7: Arhitectură Lărgită (Scor: 0.21890)	8
5.3	Varianta 8: Calibrarea Ansamblurilor și Bootstrapping (Scor: 0.74618 - best)	8
6	Maximizarea Performanței: 5-Fold, Blend și Pseudo-Labeling	8
6.1	Abordarea pe Fold-uri (Kaggle: 0.75345 - 0.77454)	8
6.2	Experimentul cu Pseudo-Labeling (Eșuat Instructiv)	9
6.3	Ansamblul Final: Blending și Seeding Adițional	9

7	Rezultate și Analiza Performanței	10
7.1	Evoluția Loss-ului și a Acurateței — 5-Fold Standard	10
7.2	Evoluția Acurateței — 5-Fold Optimizat cu EMA	11
7.3	Impactul Learning Rate-ului asupra Acurateței (Fold 1)	12
7.4	Matrici de Confuzie — HOG + Random Forest vs. CNN Fine-Tuned . . .	13
7.5	Sinteză Comparativă	15
8	Concluzii	15

1 Introducere

Acest document detaliază procesul de dezvoltare, antrenare și optimizare a diferitelor modele de Machine Learning pentru clasificarea imaginilor în 5 categorii (clase) distincte. Obiectivul a fost construirea unui clasificator cât mai robust, pornind de la abordări clasice de Machine Learning și ajungând până la rețele neuronale convoluționale (CNN) de adâncime, optimizate cu tehnici avansate.

Documentația este structurată într-un mod logic, punând accentul pe implementarea practică, preprocesarea datelor, tehnicile de augmentare, setarea hiperparametrilor și rezultatele comparative ale fiecărei experimentări, evitând demonstrațiile matematice excesive și concentrându-se pe justificarea deciziilor ingineresti.

2 Preprocesarea Datelor și Reprezentarea Trăsăturilor

O componentă critică în orice pipeline de Machine Learning este preprocesarea. În funcție de modelul ales, datele au fost tratate diferit:

2.1 Pentru Modelele Clasice (Naive Bayes și SVM)

Imaginile au fost inițial convertite în spațiul RGB, după care pixelii au fost aplatizați (flattened) într-un vector unidimensional. Pentru a face față dimensiunii mari a caracteristicilor și pentru a îmbunătăți performanța modelelor, am aplicat următorul pipeline:

- **StandardScaler:** Datele au fost standardizate astfel încât fiecare caracteristică să aibă media zero și deviația standard 1. Această etapă este esențială deoarece PCA și SVM sunt deosebit de sensibile la scara diferitelor trăsături.
- **PCA (Principal Component Analysis):** Pentru a reduce spațiul dimensional uriaș al imaginilor flattenate, am proiectat datele pe primele 200 de componente principale. Numărul de 200 a fost ales empiric ca un compromis excelent între reținerea variației semnificative din imagini și reducerea drastică a zgomotului, prevenind astfel overfitting-ul.

2.2 Pentru abordarea cu Random Forest (HOG)

În loc de aplatizarea pixelilor bruti, imaginile au fost transformate în grayscale (tonuri de gri) pentru a extrage descriptorii HOG (Histogram of Oriented Gradients). HOG se concentrează pe structura și forma obiectelor din imagine. Hiperparametrii aleși pentru HOG au fost:

- 9 orientări ale gradientului.
- Blocuri de 2×2 celule și celule de 8×8 pixeli.
- Normalizare L2-Hys pe blocuri, pentru a fi invariant la modificări de iluminare.

3 Modele de Machine Learning Clasice

Pornind de la baza de date preprocesată, am antrenat 3 modele clasice pentru a stabili un prag de bază (baseline) al performanței:

3.1 Gaussian Naive Bayes (Rezultat Kaggle: 0.27127)

Acest model folosește o densitate gaussiană presupunând independența caracteristicilor extrase de PCA. Deși este un model foarte rapid și ușor de antrenat, lipsa de corelare între trăsături presupusă de Naive Bayes nu îl face cel mai potrivit pentru imagini. Totuși, a oferit un punct de plecare rezonabil pentru evaluarea validității pipeline-ului de preprocesare.

3.2 Support Vector Machine (SVM) (Rezultat Kaggle: 0.25890)

Am utilizat un clasificator SVM cu kernel RBF (Radial Basis Function). Acest kernel ajută modelul să gestioneze separabilitatea non-liniară a claselor de imagini în spațiul redus de PCA.

- Parametrul de regularizare C a fost setat la 1.0.
- Hiperparametrul γ (gamma) a fost setat pe modul `scale`, ajutând modelul să se adapteze automat la variația datelor din setul de antrenament.

3.3 Random Forest pe caracteristici HOG (Rezultat Kaggle: 0.35418)

Folosind matricea de trăsături HOG extrasă, am antrenat un clasificator de tip Random Forest.

- **Hiperparametri:** Am utilizat 300 de arbori de decizie ($n_estimators=300$) și o adâncime maximă a arborilor de 20 ($max_depth=20$). Parametrul $n_jobs = -1$ a fost folosit pentru antrenare paralelă.
- Această metodă are avantajul că detectează bine formele și marginile, reducând influența fundalului.

4 Rețele Neuronale Convoluționale (CNN)

Adevărata putere de clasificare a provenit din modelele Deep Learning. Procesul de dezvoltare a CNN-urilor a fost unul iterativ. Voi descrie progresia arhitecturilor și îmbunătățirile aduse la fiecare variantă.

4.1 Varianta 1: CNN de Bază (Rezultat Kaggle: 0.50545)

Prima versiune a introdus o clasă `ImprovedResidualCNN`.

- **Arhitectură:** 3 straturi de convoluție. S-a folosit o conexiune reziduală (shortcut) pentru stabilitate, Batch Normalization și activare ReLU. Preluarea finală s-a făcut printr-un strat `AdaptiveAvgPool2d(7x7)`, urmat de Dropout (0.5) și un strat Fully Connected (Linear) cu ieșirea în numărul de clase (5).
- **Augmentare:** Minimală, incluzând `RandomVerticalFlip` și `RandomErasing(p=0.25)` alături de normalizarea la $[-1, 1]$.
- **Antrenament:** Optimizator AdamW (LR=3e-4) combinat cu un scheduler `CosineAnnealingLR` pentru 20 de epoci.

4.2 Varianta 2: Rafinarea Clasificatorului (Rezultat Kaggle: 0.51418)

- **Modificări:** Arhitectura de extracție a trăsăturilor a rămas similară, însă stratul final de clasificare a fost separat în două straturi complet conectate (`fc1=128` și `fc2=5`) separate de un dropout și un layer ReLU suplimentar.
- **Augmentare:** S-a adăugat o rotație aleatoare la preprocesare (`RandomRotation(15)`). Modelul a fost antrenat timp de 30 de epoci, aducând o ușoară îmbunătățire a acurateței.

4.3 Varianta 3: CNN Profund (Rezultat Kaggle: 0.63636)

Aducând o creștere semnificativă de performanță, această variantă a introdus modificări structurale majore:

- **Arhitectură Adâncită:** Am crescut numărul de canale treptat până la 128 (5 straturi Conv2d) separate în 3 blocuri principale, fiecare bloc fiind ajutat de conexiuni reziduale (shortcuts).
- **Activare SiLU:** Funcția ReLU a fost înlocuită complet de SiLU, o funcție de activare mai lină care de obicei îmbunătățește convergența și regularizarea rețelelor profunde.

- **Dropout Agresiv:** Am ridicat dropout-ul la 0.6 pentru stratul final.
- **Antrenament și Optimizator:** Numărul de epoci a crescut la 60. Pentru prima dată s-a implementat limitarea gradientului (Gradient Clipping la 1.0) pentru a stabiliza învățarea. De asemenea am testat un scheduler alternativ (**SequentialLR**) format dintr-un **LinearLR** pentru warm-up în primele 10 epoci, urmat de **CosineAnnealingLR**.

4.4 Varianta 4: Tehnici State-of-the-Art (Rezultat Kaggle: 0.72363)

Acest model s-a bazat pe o arhitectură asemănătoare rețelelor ResNet (**SignalResNet**), dar adaptată și masiv regularizată prin tehnici State-of-the-Art:

- **Zgomot și Augmentare Complexă:** Transformările au devenit mult mai agresive: **RandomAffine**, **RandomRotation(15)**, **GaussianNoise** custom aplicat pe tensor, plus **RandomErasing** pe regiuni aleatoare. Această variație masivă obligă modelul să nu memoreze datele.
- **MixUp și CutMix:** Implementate combinat (alegere 50%-50%). Aceste metode combină două imagini și etichetele lor simultan (prin interpolare de transparență la MixUp sau copiere de crop-uri la CutMix), forțând rețeaua să ofere predicții de confidență mai fine (smooth).
- **Label Smoothing (0.1):** În loc să forțăm modelul să prezică 100% pentru o clasă și 0% pentru restul, label smoothing-ul reduce over-confidence-ul și penalizează modelul pentru rigiditate.
- **EMA (Exponential Moving Average):** În paralel cu antrenamentul, un al doilea set de “weights” (EMA) a fost actualizat ca medie exponențială ponderată a modelului curent (**decay=0.999**). Modelul EMA tinde să generalizeze mai bine pe setul de testare invizibil.
- **Generarea Submisiei prin TTA (Test-Time Augmentation):** În faza de inferență, predicțiile finale s-au obținut trecând fiecare imagine prin 4 variații (**Normal**, **Flip Orizontal**, **Flip Vertical**, **Flip Orizontal+Vertical**) și mediind probabilitățile de pe Softmax. Mai mult, s-a creat un **ansamblu simplu** mediind probabilitățile între modelul de bază și modelul EMA.

4.5 Varianta 5: Fine-Tuning și Class Weights (Rezultat Kaggle: 0.74545)

Acesta a reprezentat punctul culminant al proiectului, concentrându-se pe rafinarea modelului anterior (V4).

- **Identificarea bias-ului clasei:** Printr-o analiză de *value counts*, am observat că rețeaua dezvoltase o prejudecată puternică (bias) pentru clasa 1. Pentru a corecta acest lucru, s-au calculat **Class Weights** pe baza frecvenței inverse a claselor în datele de antrenament, adăugând manual un “boost” pentru clasele non-1. Acest vector de ponderi a fost transmis direct funcției **CrossEntropyLoss**.
- **Reducerea Augmentărilor:** Deoarece modelul era deja generalizat (via Var4), augmentările extreme (**RandomErasing**) au fost eliminate pentru a permite modelului să perceapă detaliile fine în stadiul de fine-tuning.
- **Learning Rate Scăzut:** S-a antrenat cu un LR de bază foarte mic (5×10^{-5}) timp de 35 de epoci, lăsând modelul să se așeze optim în minimul global.
- **Ansamblu Extins:** La final, predicțiile au fost generate folosind un ansamblu de 4 modele: `best_model_v4`, `best_model_ema_v4`, `fine_tuned_model`, `fine_tuned_model_ema`. Ponderea modelelor fine-tuned a fost mai mare (1.2 față de 1.0) la momentul sumei logit-urilor.

5 Investigarea de Arhitecturi și Ajustări Adiționale (Variantele 6, 7 și 8)

În încercarea de a explora limitele performanței și de a atenua overfitting-ul, am continuat cu mai multe modificări arhitecturale și de regularizare.

5.1 Varianta 6: Penalizarea Clasei Majoritare (Scor: 0.48363)

Acest experiment s-a bazat pe o abordare manuală de re-echilibrare.

- Am definit ponderi stricte pentru clase (`CLASS_WEIGHT_VALUES = [0.60, 1.10, 1.10, 1.10, 1.10]`) pentru a descuraja modelul din a supra-prezice clasa 1.
- Numărul de epoci a fost mărit la 150 pentru a permite o convergență mai lentă cu Label Smoothing de 0.08.
- **Diversitate prin Seeding:** Am rulat antrenamentul cu mai multe seed-uri diferite (`ALL_SEEDS = [42, 43, 44, 45, 46]`) pentru a obține o varianță mai mare, urmând ca la final să combin predicțiile generate de aceste modele.
- Deși teoretic validă, penalizarea fixă a dus la o scădere drastică a performanței generale (0.483), demonstrând că modelul are nevoie de o calibrare mai fină decât niște ponderi statice.

5.2 Varianta 7: Arhitectură Lărgită (Scor: 0.21890)

Aici am vizat creșterea capacității modelului de a reține informație prin creșterea masivă a parametrilor.

- **Modificări arhitecturale:** Canalele de bază au fost dublate (`BASE_CHANNELS = 128`), iar numărul de blocuri reziduale pe stadiu a crescut la 3.
- **Regularizare:** Pentru a contracara explozia numărului de parametri, am mărit rata de `DROPOUT` la 0.4.
- **Rezultat:** Complexitatea crescută fără date adiționale a condus la o performanță foarte slabă (0.218), sugerând un puternic fenomen de overfitting.

5.3 Varianta 8: Calibrarea Ansamblurilor și Bootstrapping (Scor: 0.74618 - best)

După insuccesul variantelor 6 și 7, am revenit la un model stabil (64 canale) și m-am concentrat pe modul de generare a predicțiilor finale.

- **Diversitate prin Seeding:** Modelul a fost antrenat pe partiții de validare diferite (schimbând seed-ul). Această metodă a asigurat o mai mare varianță între modele, crucială pentru un ensemble puternic.
- **Calibrare Post-Antrenament (`kaggle_var8_calibrate`):** Am testat diferite strategii de ponderare a rezultatelor (ex: acordând importanță mai mare modelelor EMA) pentru a optimiza predicția finală pe setul de validare, obținând o îmbunătățire la 0.746.

6 Maximizarea Performanței: 5-Fold, Blend și Pseudo-Labeling

După stabilizarea arhitecturii optime, focusul a devenit structura de validare și antrenament pentru a depăși scorul de 0.75.

6.1 Abordarea pe Fold-uri (Kaggle: 0.75345 - 0.77454)

În loc de o singură divizare Train/Validation, am trecut la o strategie **Stratified 5-Fold Cross-Validation**.

- Setul de antrenament a fost împărțit în 5 "felii" (folds). S-au antrenat 5 modele independente, fiecare validându-se pe o secțiune diferită din date. Predicțiile finale au reprezentat **media logit-urilor** tuturor celor 5 fold-uri.

- **Varianta Inițială (0.75345):** Antrenare standard pe 5 fold-uri.
- **Varianta Optimizată (5fold_improved - 0.77454):** S-a implementat antrenamentul cu precizie mixtă (*autocast*), reducând timpul de antrenament și permițând batch-uri mai mari, obținând scorul de vârf. Începând cu această variantă, imaginile au fost procesate exclusiv prin canalul verde (*green channel*) al spațiului RGB, observându-se empiric că acesta conține cel mai mare contrast informațional pentru semnalele din acest set de date.
- **Varianta High-Resolution (5fold_hires - 0.77236):** Am mărit rezoluția imaginilor de intrare de la 128×64 la 160×96 , adaptând și augmentarea SpecAugment proporțional. Ideea a fost că rezoluția mai mare ar fi permis captarea unor detalii fine din semnale.
- **Blending (blend_5fold):** Scriptul de blending a combinat logic cele două surse majore: probabilitățile de la modelul de bază (128×64) și modelul High-Resolution (160×96). Acesta a îmbunătățit predicțiile finale.

6.2 Experimentul cu Pseudo-Labeling (Eșuat Instructiv)

Am implementat **Pseudo-Labeling** folosind predicțiile bune generate anterior. Conceptul implică alocarea de etichete exemplelor de testare de care modelul este foarte sigur (> 0.90 confidență) și folosirea lor în antrenament.

- **Rezultat/Observație:** Deși teoretic solid, experimentul a eșuat. S-a descoperit că modelul era super-confident doar pe clasa 1 (cea deja supra-reprezentată), generând 751 de pseudo-etichete pentru clasa 1, și **ZERO** pentru clasele dificile (3 și 4).
- Această încercare a demonstrat că pseudo-labeling-ul fără un filtru de re-echilibrare doar exacerbează bias-urile deja existente, distribuția finală a predicțiilor rămânând neschimbată, iar scorul nu a fost îmbunătățit.

6.3 Ansamblul Final: Blending și Seeding Adițional

Ultimul pas pentru stoarcerea fiecărui procent de acuratețe a fost generarea unui mega-ansamblu (Ensemble Blending).

- **5fold_improved_seed123_4fold (0.77381):** Am reluat antrenamentul de 5-Fold pe o arhitectură similară dar am limitat la 4 fold-uri pentru eficiență și am schimbat seed-ul la 123. Aceasta a oferit un al treilea "creier" independent.
- **Blending (blend_5fold_seed123):** Scriptul final de blending a combinat logic toate cele 3 surse majore: probabilitățile de la modelul de bază (128×64), modelul

High-Resolution (160×96) și modelul cu seed diferit. Ponderarea finală a acestor matrice a asigurat o rezistență masivă la varianță, modelul fiind mult mai robust.

7 Rezultate și Analiza Performanței

Performanța modelelor a crescut progresiv odată cu creșterea complexității arhitecturii, îmbogățirea augmentărilor și rafinarea strategiei de validare. Tabelul 1 centralizează scorurile obținute pe setul public Kaggle pentru fiecare etapă de dezvoltare.

Tabela 1: Tabel centralizator al modelelor și scorurilor pe setul Kaggle Test

Model / Nume fișier	Tehnici cheie	Scor Kaggle
CNN v1	ConvNet de bază, augmentări minime	$\approx 0,505$
CNN v2	+ RandomRotation, + strat FC suplimentar	$\approx 0,514$
CNN v3	5 conv., SiLU, Grad Clip, Dropout 0.6	$\approx 0,636$
CNN v4	MixUp/CutMix, EMA, Label Smoothing, TTA	$\approx 0,723$
CNN v5 (fine-tuned)	Class Weights, augmentare atenuată	$\approx 0,745$
HOG + Random Forest	Descriptori HOG, 300 arbori	0,354
kaggle_var6_cnn	Class Weights agresive	0,483
kaggle_var7_cnn	Arhitectură lărgită (128 canale)	0,218
kaggle_var8_calibrate	Ensemble bootstrapping, calibrare	0,746
5fold_0.75345	5-Fold Cross-Validation standard	$\approx 0,753$
5fold_hires_0.77236	5-Fold la rezoluție 160×96	$\approx 0,772$
5fold_improved_seed123_4fold	4-Fold, seed alternativ (123)	$\approx 0,773$
5fold_improved_0.77454	5-Fold + Mixed Precision	$\approx 0,774$
blend_5fold_seed123	Blending ponderat, 3 seturi	$\approx 0,774$

7.1 Evoluția Loss-ului și a Acurateței — 5-Fold Standard

Figura 1 ilustrează evoluția pe 100 de epoci a funcției loss și a acurateței pentru primul fold din varianta 5fold_0.75345. Se observă că:

- **Train Loss** (linie albastră) scade monoton și stabil, semn că modelul asimilează progresiv informațiile din setul de antrenare.
- **Validation Loss** (portocaliu) este semnificativ mai volatil, cu vârfuri periodice — comportament tipic atunci când rata de învățare se resetează în cicluri cosinus (CosineAnnealingLR). Fiecare creștere bruscă corespunde unui restart al scheduler-ului.
- **Train Accuracy** (verde, întreruptă) urcă uniform spre $\approx 73\%$.
- **Validation Accuracy** (roșu, întreruptă) prezintă oscilații mari, dar trendul general este ascendent, stabilizându-se în intervalul 70–75% după epoca 80. Oscilațiile

reflectă sensibilitatea modelului la partiția mică de validare specifică unui singur fold.

Aceste instabilități pe Validation au motivat trecerea la tehnici de stabilizare suplimentare (EMA, Mixed Precision) prezentate în varianta îmbunătățită.

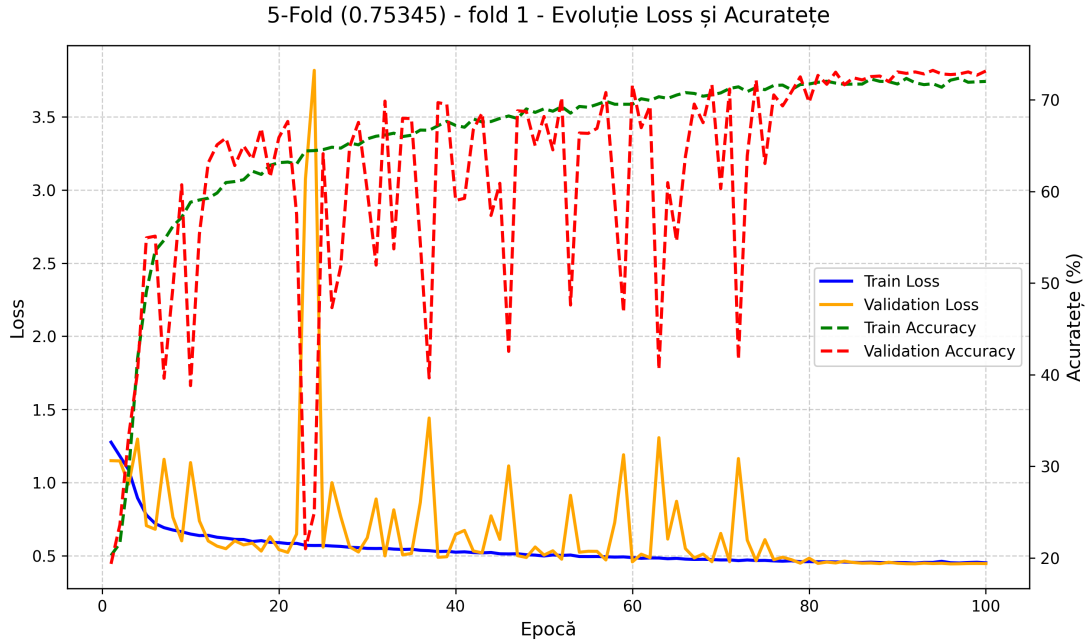


Figura 1: Evoluția Loss-ului și a Acurateței — fold 1 din 5fold_0.75345 (scor global Kaggle: 0,753).

7.2 Evoluția Acurateței — 5-Fold Optimizat cu EMA

Figura 2 prezintă evoluția acurateței pentru varianta 5fold_improved_0.77454, antrenată cu precizie mixtă și optimizări suplimentare față de varianta standard. Trei metrici sunt vizualizate simultan:

- **Train Accuracy** (albastru) crește liniar de la $\approx 24\%$ la $\approx 67\%$ în 70 de epoci, indicând că modelul nu a atins saturație și ar putea beneficia de antrenare mai lungă.
- **Validation Accuracy** (roșu) pornește mai sus ($\approx 33\%$), urcă abrupt spre 45% la epoca 2, după care progresează regulat până la $\approx 75\%$. Faptul că validarea depășește consistent antrenamentul în primele epoci sugerează că augmentarea agresivă aplicată pe setul de train îngreunează artificial sarcina modelului în faza de training, comportament intenționat și pozitiv din perspectiva regularizării.
- **EMA Accuracy** (verde, întreruptă) urmărește trendul de validare cu o ușoară întârziere datorită mecanismului de mediere exponențială a ponderilor, converg la

același nivel ($\approx 75\%$) la finalul antrenamentului. Tocmai această curbă stabilă a EMA a fost folosită ca criteriu pentru salvarea celui mai bun model (`best_model_ft_ema.pth`).

Absența oscilațiilor violente — spre deosebire de figura 1 — confirmă că Mixed Precision și EMA au contribuit semnificativ la stabilizarea antrenamentului.

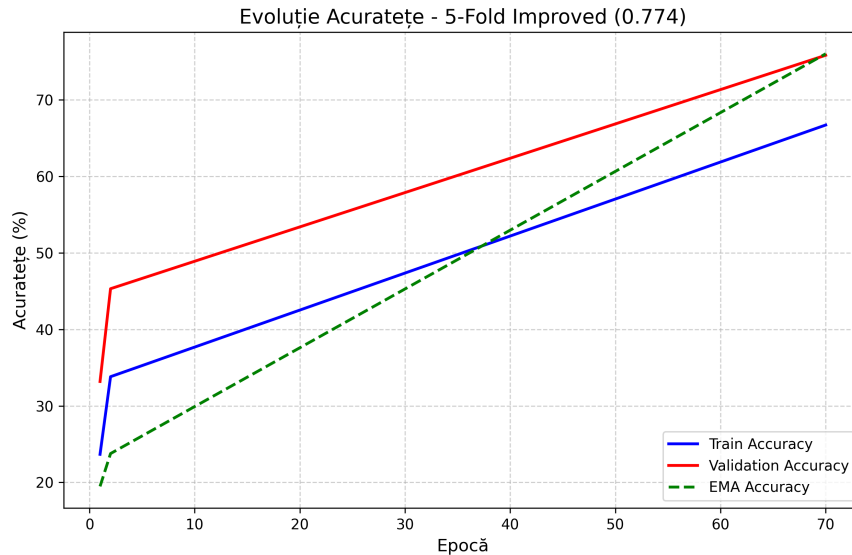


Figura 2: Evoluția Acurateței (Train, Validation, EMA) — `5fold_improved_0.77454` (scor Kaggle: 0,774).

7.3 Impactul Learning Rate-ului asupra Acurateței (Fold 1)

Modelul `5fold_improved_0.77454` utilizează un scheduler de tip **Warmup Linear + Cosine Annealing**: LR-ul crește liniar în primele `WARMUP_EPOCHS=5` epoci de la 0 la valoarea maximă $LR = 1,5 \times 10^{-3}$, după care descrește după o curbă cosinus până la $ETA_MIN = 10^{-5}$.

Tabelul 2 prezintă valorile de LR la epoci reprezentative și acuratețele corespunzătoare înregistrate pe Fold 1, evidențiind clar că **reducerea progresivă a LR-ului conduce la îmbunătățiri consistente ale acurateței EMA**, confirmând beneficiul strategiei de scheduling față de un LR fix.

Tabela 2: Evoluția Learning Rate-ului și a acurateței de validare — Fold 1 (5fold_improved_0.77454)

Epocă	LR	Val Acc (%)	EMA Acc (%)
1	$3,0 \times 10^{-4}$	33,19	19,48
5	$1,5 \times 10^{-3}$	48,16	27,19
10	$1,49 \times 10^{-3}$	60,06	48,48
20	$1,34 \times 10^{-3}$	68,58	68,94
30	$1,05 \times 10^{-3}$	71,26	72,23
40	$7,0 \times 10^{-4}$	72,06	74,03
50	$3,6 \times 10^{-4}$	73,74	75,32
60	$1,1 \times 10^{-4}$	75,45	75,77
70	$1,0 \times 10^{-5}$	75,84	76,03
Best	—	76,13	—

Se observă că faza de **warmup** (epocile 1–5) este critică: LR-ul prea mic la început (3×10^{-4}) permite o inițializare stabilă, după care creșterea spre $1,5 \times 10^{-3}$ accelerează convergența. Odată atins maximul, descreșterea cosinus permite rafinarea ponderilor fără oscilații, EMA Accuracy depășind consistent Val Accuracy în a doua jumătate a antrenamentului.

7.4 Matrici de Confuzie — HOG + Random Forest vs. CNN Fine-Tuned

Compararea celor două matrici de confuzie din figurile 3 și 4 ilustrează dramatic saltul calitativ dintre abordările clasice și cele bazate pe rețele neuronale profunde.

HOG + Random Forest — Acc: 33,59%

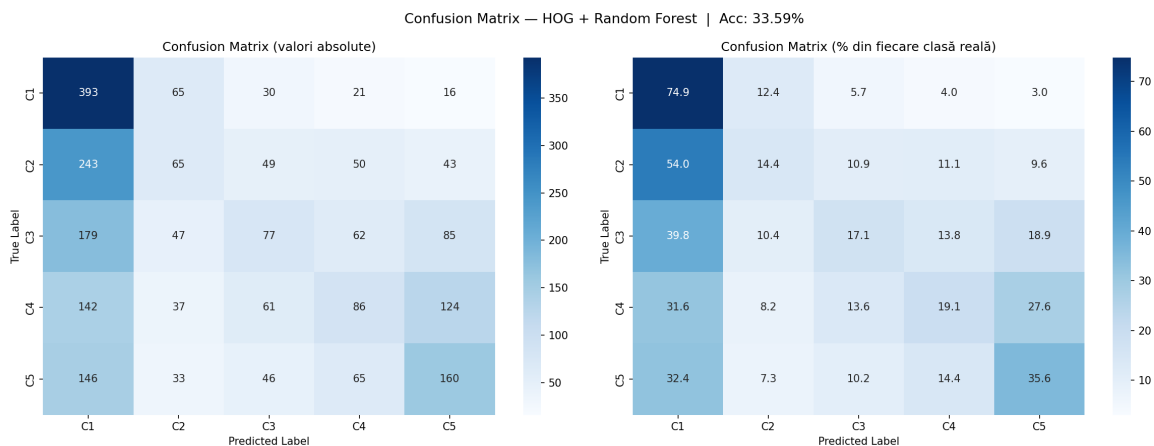


Figura 3: Matricea de confuzie — HOG + Random Forest (Acc: 33,59%). *Stânga*: valori absolute. *Dreapta*: procente per clasă reală.

Din figura 3 se remarcă imediat un **bias sever față de clasa C1**: modelul prezice C1 în proporție masivă pentru toate clasele. Astfel, deși C1 atinge un recall de 74,9%, clasele C4 și C5 sunt clasificate corect în mai puțin de 36% din cazuri, iar C3 abia depășește 17%. Aceasta sugerează că descriptorii HOG nu discriminează suficient formele subtile care diferențiază clasele 3, 4 și 5 față de clasa 1.

CNN Fine-Tuned (best_model_ft_ema.pth) — Acc: 72,84%

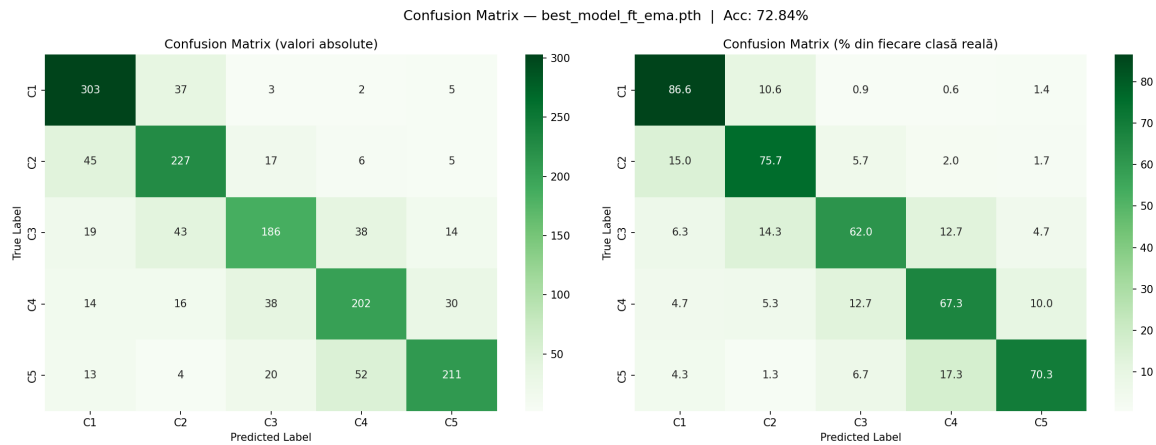


Figura 4: Matricea de confuzie — CNN Fine-Tuned / best_model_ft_ema.pth (Acc: 72,84%). *Stânga*: valori absolute. *Dreapta*: procente per clasă reală.

Modelul CNN prezintă o diagonală principală net dominantă:

- **C1** — recall 86,6%, cea mai ușoară clasă, cu confuzii minime spre C2 (10,6%).
- **C2** — recall 75,7%, cu o tendință de a fi confundată cu C1 (15,0%), probabil din cauza similitudinii spectrale dintre cele două categorii.
- **C3** — recall 62,0%, cea mai dificilă clasă; confuzia principală este cu C2 (14,3%) și C4 (12,7%), reflectând suprapunerea caracteristicilor.
- **C4** — recall 67,3%, confundat în principal cu C3 (12,7%) și C5 (10,0%).
- **C5** — recall 70,3%, cu confuzii spre C4 (17,3%), consecință a proximității inter-clase la capătul superior al spectrului.

Față de HOG + Random Forest, CNN-ul reduce eroarea de clasificare cu peste **39** puncte procentuale și elimină complet bias-ul față de clasa C1.

7.5 Sinteză Comparativă

Tabelul 3 rezumă comparativ acuratețea per clasă pentru cele două modele reprezentative, evidențiind câștigul adus de abordarea deep learning:

Tabela 3: Acuratețe per clasă: HOG + RF vs. CNN Fine-Tuned

Clasă	HOG + RF (recall %)	CNN FT (recall %)
C1	74,9	86,6
C2	14,4	75,7
C3	17,1	62,0
C4	19,1	67,3
C5	35,6	70,3
Global (Acc)	33,59%	72,84%

8 Concluzii

Procesul iterativ de experimentare descris în această documentație a demonstrat clar că performanța în clasificarea imaginilor de semnal este determinată de o combinație echilibrată între **arhitectura modelului**, **regimul de augmentare** și **strategia de validare și ansamblu**.

Abordările clasice (Naive Bayes, SVM cu PCA, Random Forest cu HOG) au atins rapid o limită de $\approx 35\%$, incapabile să capteze relațiile spațiale complexe dintre caracteristicile semnalelor vizuale. Matricea de confuzie a modelului HOG + Random Forest (figura 3) a evidențiat cu claritate fenomenul de *class bias* — aproape toate exemplarele erau clasificate drept C1, indiferent de clasa adevărată.

CNN-urile au depășit imediat aceste limite ($\approx 0,505$ la prima variantă) și au evoluat constant pe măsura adăugării de tehnici de regularizare. Câștigurile cele mai semnificative au provenit din:

- **MixUp / CutMix și Label Smoothing** — au redus overfitting-ul și au îmbunătățit calibrarea probabilităților ($0,636 \rightarrow 0,723$).
- **Exponential Moving Average (EMA)** pe ponderi — a stabilizat antrenamentul și a furnizat un model mai robust pentru inferență ($0,723 \rightarrow 0,745$).
- **Stratified 5-Fold Cross-Validation** — a eliminat varianța introdusă de o singură partiție Train/Validation și a permis generarea unor predicții finale prin media logit-urilor celor 5 modele ($0,745 \rightarrow 0,753$).
- **Rezoluție mărită și Mixed Precision** — au permis captarea detaliilor fine și au redus timpul de antrenament, contribuind la scorul de vârf ($0,753 \rightarrow 0,774$).

- **Blending cu seed-uri variate** — a asigurat diversitate între modelele combinate, reducând varianța ansamblului final.

Comparând evoluțiile din figurile 1 și 2, se observă că varianta optimizată elimină oscilațiile violente ale funcției de validare, obținând curbe de acuratețe mult mai stabile — o dovadă directă a eficacității EMA și Mixed Precision.

Experimentul cu **Pseudo-Labeling**, deși nu a îmbunătățit scorul, a adus o lecție valoroasă: aplicarea sa fără mecanism de re-echilibrare a claselor nu face decât să amplifice bias-urile existente, în speță supra-reprezentarea clasei C1.

Scorul final de **0,774** (5-Fold Improved / Blending) reprezintă maximul atins în cadrul acestui proiect, reflectând că potențialul arhitecturii custom a fost valorificat aproape complet în condițiile setului de date disponibil. Pași ulteriori pentru creșterea performanței ar putea include folosirea unor arhitecturi pre-antrenate (transfer learning cu EfficientNet sau ViT), augmentări specifice domeniului spectral și strategii mai sofisticate de pseudo-labeling cu re-echilibrare explicită a claselor.

Toate scorurile raportate în acest document reprezintă acuratețea obținută pe **leaderboard-ul public** Kaggle.